

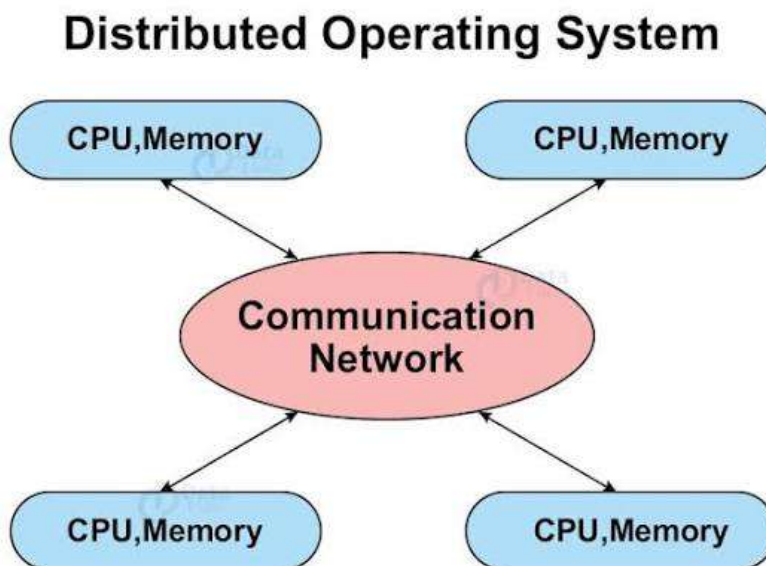
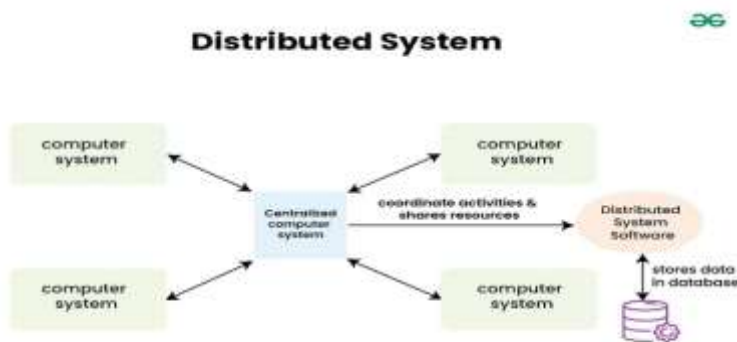
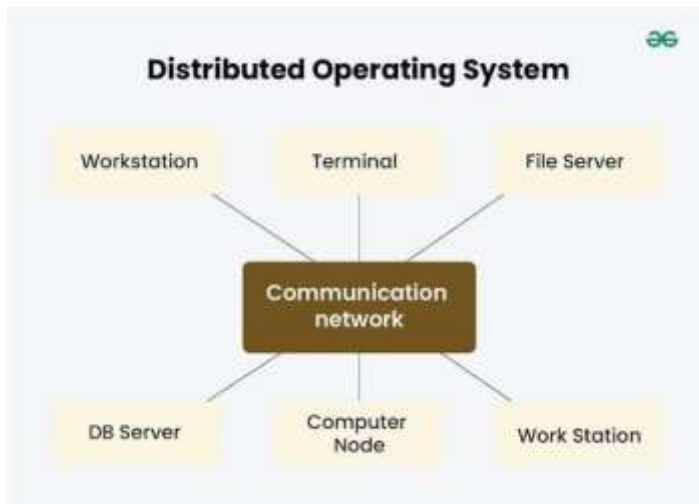
## UNIT-3

### Distributed Operating System (DOS)

A Distributed Operating System (DOS) is a specialized software model that manages a collection of independent, networked computers and makes them appear to users as a single, unified system. Unlike a traditional OS that resides on one machine, a DOS is split into sections and loaded across multiple nodes, allowing them to communicate and share resources seamlessly.

#### Key Characteristics

- **Transparency:** The system's complexity is hidden; users perceive the entire network as one computer, unaware of where files or processes are physically located.
- **Scalability:** New machines (nodes) can be added to the network to increase capacity without significantly affecting performance or requiring infrastructure changes.
- **Fault Tolerance:** Because tasks and data are distributed, the failure of a single node does not bring down the entire system; other nodes can pick up the workload.
- **Resource Sharing:** It enables secure sharing of hardware (printers, CPUs), software, and data across all connected sites.



#### Types of Distributed Systems

1. **Client-Server:** Clients request resources or services, and dedicated servers provide them. These are often considered "tightly coupled" because of the centralized server role.
2. **Peer-to-Peer (P2P):** All nodes are equal and can act as both client and server. There is no central authority, making the system "loosely coupled" and highly resilient.

3. **Middleware:** A software layer that sits between the OS and applications, enabling different programs to communicate across heterogeneous environments.
4. **Multi-tier (3-tier/N-tier):** Separates functions (like presentation, business logic, and data) into different layers, often used in large-scale web applications.

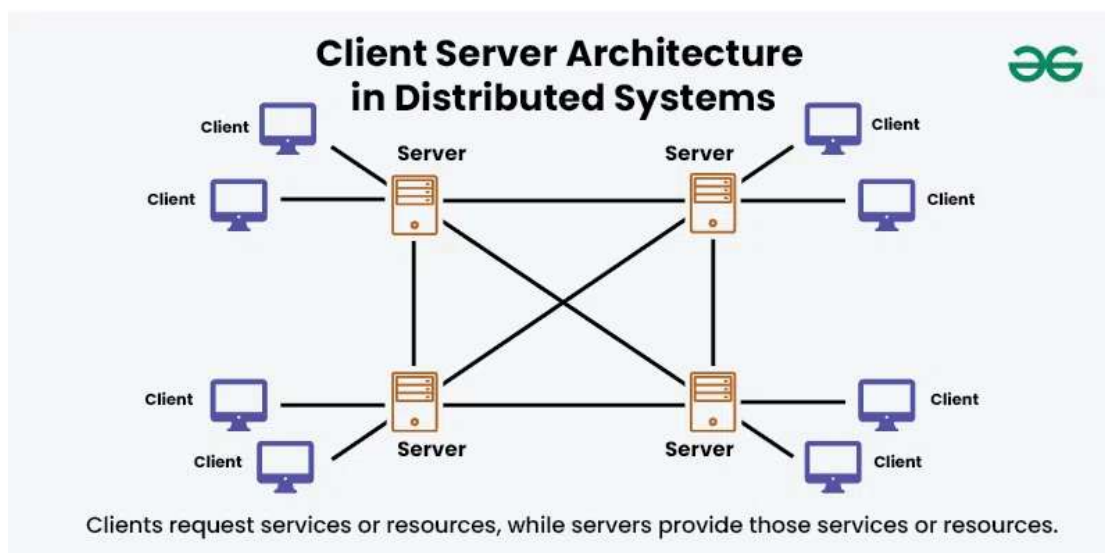
### Common Examples

- **Solaris:** Designed for SUN multiprocessor workstations.
- **Mach/OS:** A multitasking and multithreading UNIX-compatible system.
- **Plan 9:** Developed at Bell Labs to make a network of computers appear as one.

### Practical Applications

- **Cloud Computing:** Platforms like AWS, Microsoft Azure, and Google Cloud rely on distributed architectures for scalable storage and compute power.
- **Telecommunications:** Used in cellular networks, wireless sensor networks, and complex routing algorithms.
- **Real-Time Systems:** Critical for aircraft control and medical imaging where immediate response times across multiple processors are necessary.
- **Big Data:** Systems like Apache Hadoop use distributed frameworks to process massive datasets across clusters

### Client–Server Architecture



In the **Client–Server model**, clients request services and servers provide services.

### Working

1. Client sends request
2. Server processes request
3. Server sends response

### Example

- Web browser (client) → Web server
- Student system accessing database server

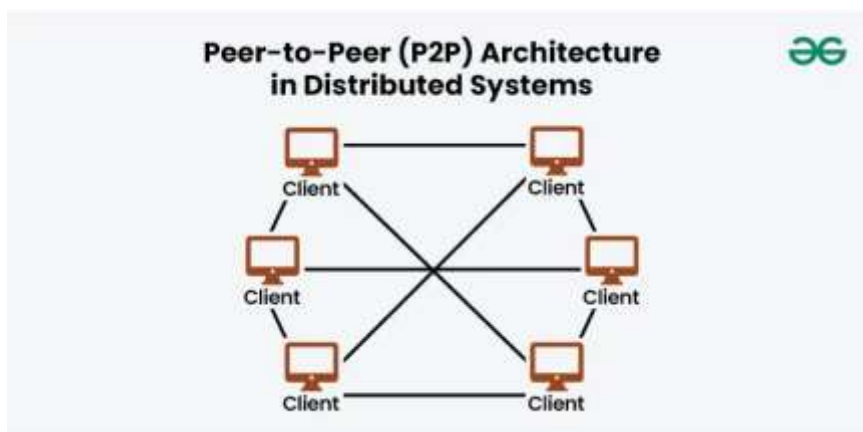
### Advantages

- Simple design
- Easy to manage and secure
- Centralized control

### Disadvantages

- Server is single point of failure
- Scalability problems

### Peer-to-Peer (P2P) Architecture



All nodes act as **both clients and servers**.

### Example

- BitTorrent
- Blockchain networks

### Advantages

- No central failure

- Highly scalable

#### **Disadvantages**

- Difficult management
- Security issues
- 

#### **Design Issues in Distributed Operating Systems**

Design issues address **how to make distributed systems efficient, reliable, and user-friendly.**

##### **1. Transparency**

System hides complexity from users.

##### **Types of Transparency**

1. **Location transparency** – resource location hidden
2. **Access transparency** – same operations for local/remote

##### **Example**

User accesses a file without knowing where it is stored.

##### **2. Scalability**

System should work efficiently as:

- Number of users increases
- Number of machines increases
- Geographic area increases

##### **3. Fault Tolerance**

System continues working even if some nodes fail.

##### **Techniques**

- Replication
- Checkpointing
- Recovery mechanisms

##### **4 Concurrency**

Multiple processes execute at the same time without interference.

## 5 Security

Includes:

- Authentication
- Authorization
- Encryption
- Secure communication

### Communication Models in Distributed Systems

#### Introduction

In a **distributed system**, processes are located on different machines and **do not share memory**. Hence, they communicate using **message passing**.

A **communication model** defines **how data is exchanged** between processes.

---

#### List of Communication Models

1. Message Passing Model
  2. Client–Server Communication Model
  3. Remote Procedure Call (RPC) Model
  4. Message-Oriented Communication Model
  5. Stream-Oriented Communication Model
  6. Publish–Subscribe Model
- 

#### 1. Message Passing Model

Processes communicate by explicitly **sending and receiving messages**.

##### Basic Operations

- send(destination, message)
- receive(source, message)

##### Types

- **Blocking (Synchronous)**
- **Non-blocking (Asynchronous)**

##### Example

- MPI (Message Passing Interface)
- Distributed chat systems

##### Advantages

- Simple and flexible
- Suitable for distributed environments

#### **Disadvantages**

- Programmer handles communication explicitly
- Message overhead

### **2. Client–Server Communication Model**

Clients request services and servers provide services.

#### **Working**

1. Client sends request
2. Server processes request
3. Server sends reply

#### **Example**

- Web browser and web server
- Email systems

#### **Advantages**

- Easy to implement
- Centralized management

#### **Disadvantages**

- Server bottleneck
- Single point of failure

### **3. Remote Procedure Call (RPC) Model**

RPC allows a program to call a procedure on a **remote machine** as if it were a **local function call**.

#### **Working Steps**

1. Client calls local stub
2. Stub sends request to server
3. Server executes procedure
4. Result returned to client

#### **Example**

- gRPC

- NFS (Network File System)

**Advantages**

- Hides communication details
- Simplifies programming

**Disadvantages**

- Failure handling is complex
- Performance overhead